

Mục tiêu

- Thao tác với mảng 1 chiều và 2 chiều

Yêu cầu

- Làm tất cả các bài trên trong 1 Solution và nộp bài theo quy định trên Moodle.
- **KHÔNG được sử dụng vòng lặp lồng nhau (nested loop) cho cùng 1 chiều và các hàm pow(), min(), max().**
- Mỗi bài làm trong 1 project.
- Bài A: 6 bài với có ít nhất 3 bài trong 1-10 và 2 bài trong 11-20.
- Bài H: làm đủ 25 bài.

Bài tập

Bài 1 – 10

10 bài trong “**Bài tập mảng**”: P01, P02, P04, P05, P06, P09, P12, P15, P25, P26.

Bài 11 – 20

10 bài trong “**Bài tập mảng 2 chiều**”: P320, P398, P342, P347, P402, P423, P438, P457, P458, P471.

Bài 21

- Khai báo kiểu dữ liệu đa thức (SDaThuc)
- Nhập/Xuất đa thức
- Tính tổng, hiệu, tích hai đa thức
- Tính đạo hàm cấp 1 của đa thức
- Tính đạo hàm cấp k của đa thức
- Tính giá trị đơn thức tại $x = x0$

Bài 22

- Nhập/Xuất n phân số
- Rút gọn mọi phân số
- Đếm số lượng phân số âm/dương trong mảng
- Tìm phân số dương đầu tiên trong mảng
- Tìm phân số nhỏ nhất/lớn nhất trong mảng
- Sắp xếp mảng tăng dần/giảm dần

Bài 23

- Nhập/Xuất n điểm
- Đếm số lượng điểm có hoành độ dương
- Đếm số lượng điểm không trùng với các điểm khác trong mảng
- Tìm điểm có hoành độ lớn nhất/nhỏ nhất
- Tìm điểm gần gốc tọa độ nhất

Bài 24

Định nghĩa struct SSinhVien với các dữ liệu: MSSV (int), HoTen (char[]), DiemTB (float), QueQuan (char[]) và NgaySinh (SNGay). Hoàn thành các yêu cầu sau:

- Phát sinh mảng SSinhVien
- Sắp xếp mảng SSinhVien theo QueQuan rồi theo tuổi rồi theo HoTen

Bài 25

Định nghĩa struct SLopHoc với các dữ liệu: TenLop (char[]), DSSinhVien (SsinhVien[]), SiSo (uint). Hoàn thành các yêu cầu sau:

- Thêm sinh viên vào lớp học.
- Xuất thông tin lớp học (thông tin tên lớp và sĩ số ở dòng 1, các dòng tiếp theo là danh sách sinh viên).
- Tìm các sinh viên theo tên

Tìm các sinh viên theo tuổi.

Nội dung

Mảng 1 chiều

Duyệt mảng

Chỉ số của mảng luôn bắt đầu từ 0, do đó duyệt các phần tử của mảng 1 chiều n phần tử là duyệt qua các chỉ số từ 0 -> (n-1).

```
void XuatMang(int mang[], int n)
{
    for (int i=0; i<n; i++)
    {
        printf("%d, ", mang[i]);
    }
}
```

Tìm phần tử thỏa điều kiện

Duyệt mảng và dựa vào điều kiện để xác định phần tử cần tìm

```
int TimViTriMax(int mang[], int n)
{
    int idx = 0;
    int gtMax = mang[0];
    for (int i=0; i<n; i++)
    {
        if (gtMax < mang[i])
        {
            gtMax = mang[i];
            idx = i;
        }
    }
    return idx;
}
```

```
int TimViTriSoChanCuoi(int mang[], int n)
```

```

{
    for (int i=n-1; i>=0; i--)
    {
        if (mang[i] % 2 == 0)
        {
            return i;
        }
    }
    return -1;
}

```

```

// Tìm số chẵn lớn nhất nhỏ hơn mọi số lẻ trong mảng
int TimSoChanMax(int mang[], int n)
{
    int soLeMin = 0;
    for (int i=0; i<n; i++)
    {
        if (mang[i] % 2 != 0 && (soLeMin == 0 || mang[i] < soLeMin))
        {
            soLeMin = mang[i];
        }
    }
    return (soLeMin - 1);
}

```

Mảng 2 chiều

Hướng dẫn bài P346 cho các thao tác cơ bản với mảng 2 chiều.

Yêu cầu: Đếm số lượng giá trị “Hoàng Hậu” trên ma trận. Một phần tử được gọi là Hoàng Hậu khi nó lớn nhất trên dòng, trên cột và 2 đường chéo đi qua nó.

- Viết hàm xuất ma trận

```

#define MAXLEN 100

void Print2DIntArray(int a2d[][MAXLEN], const int &nRow, const int &nCol)
{
    for (int iR = 0; iR < nRow; iR++)
    {
        for (int iC = 0; iC < nCol; iC++)
        {
            cout << setw(4);
            cout << a2d[iR][iC];
        }
        cout << endl;
    }
}

```

- Viết hàm main

```

void main()
{
    srand((unsigned int)time(NULL));
    int a2d[MAXLEN][MAXLEN];
    int nRow = 5 + rand() % 10;
    int nCol = 5 + rand() % 10;
    const int maxValue = 1000;
    for (int iR = 0; iR < nRow; iR++)
    {
        for (int iC = 0; iC < nCol; iC++)
        {
            a2d[iR][iC] = rand() % maxValue;
        }
    }
    int countQueen = CountQueenValue(a2d, nRow, nCol);
    Print2DIntArray(a2d, nRow, nCol);
    cout << endl << "nQueen = " << countQueen << endl;

    system("pause");
}

```

➤ Viết hàm đếm số giá trị Hoàng Hậu

```

int CountQueenValue(int a2d[][MAXLEN], const int &nRow, const int &nCol)
{
    int count = 0;
    bool isQueen = false;
    for (int iR = 0; iR < nRow; iR++)
    {
        for (int iC = 0; iC < nCol; iC++)
        {
            isQueen = true;
            // kiểm tra các giá trị trong hàng
            for (int iCheckRow = 0; iCheckRow < nCol; iCheckRow++)
            {
                if (iCheckRow != iC && a2d[iR][iC] < a2d[iR][iCheckRow])
                {
                    isQueen = false;
                    break;
                }
            }
            if (!isQueen)
            {
                continue;
            }
            // kiểm tra các giá trị trong cột
            for (int iCheckCol = 0; iCheckCol < nRow; iCheckCol++)
            {
                if (iCheckCol != iR && a2d[iR][iC] < a2d[iCheckCol][iC])
                {
                    isQueen = false;
                    break;
                }
            }
            if (!isQueen)
            {
                continue;
            }
        }
    }
}

```

```

// kiểm tra các giá trị ở đường chéo
int iCrossColPos = iC, iCrossRowPos = iR;
int iCrossColNeg = iC, iCrossRowNeg = iR;
while (true)
{
    iCrossColPos++;
    iCrossRowPos++;
    iCrossColNeg--;
    iCrossRowNeg--;
    // nếu tất cả đều đã duyệt hết thì dừng
    if (iCrossColPos >= nRow && iCrossRowPos >= nCol && iCrossColNeg < 0 && iCrossRowNeg < 0)
    {
        break;
    }
    // kiểm tra 4 giá trị ở đường chéo
    if ((iCrossColPos < nRow && iCrossRowPos < nCol && a2d[iCrossRowPos][iCrossColPos] > a2d[iR][iC]) ||
        (iCrossColPos < nRow && iCrossRowNeg >= 0 && a2d[iCrossRowNeg][iCrossColPos] > a2d[iR][iC]) ||
        (iCrossColNeg >= 0 && iCrossRowPos < nCol && a2d[iCrossRowPos][iCrossColNeg] > a2d[iR][iC]) ||
        (iCrossColNeg >= 0 && iCrossRowNeg >= 0 && a2d[iCrossRowNeg][iCrossColNeg] > a2d[iR][iC]))
    {
        isQueen = false;
        break;
    }
}
if (!isQueen)
{
    continue;
}
count++;
}
}
return count;
}

```